

client.py

```
1  from PIL import Image, ImageTk
2  from tkinter import messagebox
3  from typing import Callable
4  from paint import Paint
5  from tkinter import ttk
6  import tkinter as tk
7  from main import app
8  import subprocess
9  import threading
10 import uvicorn
11 import sys
12 import re
13
14 class App(tk.Tk):
15     def __init__(self):
16         super().__init__()
17         self.title("Paint - Multiplayer")
18         self.icon = tk.PhotoImage(file='assets/icon.png')
19         self.iconphoto(False, self.icon)
20
21         self.resizable = True
22         self.geometry("800x600")
23
24         self.mainMenu = MainMenu(self)
25
26         self.mainMenu.pack(expand = True, fill = "both")
27
28
29         self.mainloop()
30
31     def save(self):
32         if self.paintScreen:
33             self.paintScreen.save()
34
35     def launchPaint(self, network: bool, wsUrl: str = "ws://localhost:8000"):
36         self.mainMenu.destroy()
37
38         self.paintScreen = Paint(master = self, network = network, wsUrl = wsUrl)
39
40         self.paintScreen.pack(expand = True, fill = "both")
41
42         self.bind("x", lambda _: self.paintScreen.colorBar.swapPrimarySecondary())
43
44         self.configure(menu = MenuBar(self, network))
45
46     def copyTunnelUrl(self):
47         if self.tunnelUrl:
```

```

48         self.clipboard_clear()
49         self.clipboard_append(self.tunnelUrl)
50         self.update()
51
52     def runUvicorn(self):
53         self.serverProcess = subprocess.Popen([
54             sys.executable,
55             "-m", "uvicorn",
56             "main:app",
57             "--host", "127.0.0.1",
58             "--port", "8000"
59         ])
60
61     def launchServer(self, on_tunnel_ready: Callable[[str], None]) -> None:
62
63         # Server
64         self.runUvicorn()
65
66         # Tunnel
67
68         self.tunnelProcess = subprocess.Popen(
69             [
70                 "./cloudflared.exe", "tunnel", "--url", "http://localhost:8000", "--
protocol", "http2"],
71             stdout = subprocess.PIPE,
72             stderr = subprocess.STDOUT,
73             text = True
74         )
75
76         self.tunnelUrl = None
77
78         # Stdout Thread
79         def readOutput():
80             for line in self.tunnelProcess.stdout: # type: ignore
81                 match = re.search(r"https://.*\.trycloudflare\.com", line)
82                 if match:
83                     self.tunnelUrl = match.group(0).replace("https://", "wss://")
84                     break
85
86         threading.Thread(target = readOutput, daemon = True).start()
87
88         # Loader
89         self.loadingWindow = tk.Toplevel(self)
90         self.loadingWindow.title("Starting server...")
91
92         self.loader = ttk.Progressbar(self.loadingWindow, mode = "indeterminate",
length = 300)
93         self.loader.pack(padx = 20, pady = 20)
94         self.loader.start()

```

```

95         # Polling
96
97     def poll():
98         if self.tunnelUrl:
99             self.loader.stop()
100             self.loadingWindow.destroy()
101             on_tunnel_ready(self.tunnelUrl)
102         else:
103             self.after(100, poll)
104
105     poll()
106
107
108 class MenuBar(tk.Menu):
109     def __init__(self, master: App, network: bool, *args, **kwargs):
110         super().__init__(*args, **kwargs)
111
112         self.configure(
113             background = "darkgray",
114             relief = "raised",
115             border = 2
116         )
117
118         # Sub Menus
119         self.fileMenu = tk.Menu(self, tearoff = False)
120         self.fileMenu.add_command(label = "New")
121         self.fileMenu.add_command(label = "Save", command = master.save)
122         if network:
123             self.fileMenu.add_command(label = "Copy join link.", command =
master.copyTunnelUrl)
124         self.fileMenu.add_separator()
125         self.fileMenu.add_command(label = "Exit", command = self.master.quit)
126
127         # Adding Submenus to Menubar
128         self.add_cascade(label = "File", menu = self.fileMenu)
129
130 class MainMenu(tk.Frame):
131     def __init__(self, master: App, *args, **kwargs):
132         super().__init__(master = master)
133
134         self.app: App = master
135
136         self.configure(background = "#FEFEFE")
137
138         self.logoSize = (430, 255)
139         self.logoImage = ImageTk.PhotoImage(Image.open("assets/
logo.png").resize((self.logoSize[0], self.logoSize[1])))
140
141         self.logo = tk.Label(

```

```

142         self,
143         image = self.logoImage,
144         width = self.logoSize[0],
145         height = self.logoSize[1]
146     )
147
148     self.logo.pack(expand = True)
149
150     self.buttonWrapper = tk.Frame(self, background = "#FEFEFE")
151
152     self.singlePlayer = tk.Button(
153         self.buttonWrapper,
154         text = "Single Player",
155         command = lambda: self.app.launchPaint(False)
156     )
157
158     self.singlePlayer.pack(expand = True, pady = 5)
159
160     self.multiPlayer = tk.Button(
161         self.buttonWrapper,
162         text = "Multi Player",
163         command = self.networkScreen
164     )
165
166     self.multiPlayer.pack(expand = True, pady = 5)
167
168     self.quitButton = tk.Button(
169         self.buttonWrapper,
170         text = "Quit",
171         command = self.master.quit
172     )
173
174     self.quitButton.pack(expand = True, pady = 5)
175
176     self.buttonWrapper.pack(expand = True)
177
178     def networkScreen(self):
179         for child in self.wininfo_children():
180             child.destroy()
181
182         self.hostRoomButton = tk.Button(self, text = "New Room", command =
self.hostRoom)
183         self.hostRoomButton.pack(expand = True, anchor = "s")
184
185         self.orLabel = tk.Label(self, text = "OR", font = ("TKDefaultFont", 30))
186         self.orLabel.pack(pady = 20)
187
188         self.entryLabel = tk.Label(self, text = "Existing room websocket url:")
189         self.entryLabel.pack()

```

```
190         self.urlEntry = tk.Entry(self, width = 60)
191         self.urlEntry.pack()
192         self.connectButton = tk.Button(self, text = "Connect", command =
self.connect)
193         self.connectButton.pack(expand = True, anchor = "n")
194
195     def connect(self):
196         self.app.launchPaint(wsUrl = self.urlEntry.get(), network = True)
197         self.destroy()
198
199     def hostRoom(self):
200         self.app.launchServer(self.tunnelReady)
201
202     def tunnelReady(self, tunnelUrl: str):
203         self.showUrl = messagebox.showinfo("Websocket URL", f"Your Url:
{tunnelUrl}")
204         print(tunnelUrl)
205         self.app.launchPaint(network = True, wsUrl = tunnelUrl)
206         self.destroy()
207
208 if __name__ == "__main__":
209     App()
```